

Skill: معمارية ال Backend

مبنية على تحليل ريبو ASP.NET Core حقيقي (El-Haty Restaurant Backend)

ملف مرجعي (Skill) — ضيفه لـ Antigravity قبل ما تبدأ أي فيتشر في مشروع ميزان

ليه الملف ده مهم؟

AI) اشتغل بنفس الأنماط دي من الأول، النتيجة هتكون منظمة واحترافية، وأسهل بكتير إن أي حد (حتى صاحبك) يكمل عليها لو احتجت. (لصاحبك يوسف)، واستخرجت منه الأنماط (Patterns) المستخدمة فيه. دي مش نظرية — دي كود شغال فعليا. لو (Antigravity) حلت ريبو Backend حقيقي شغال بـ ASP.NET Core

1. البنية المعمارية (Clean Architecture)

المشروع مقسم لأربع طبقات منفصلة، كل واحدة لها مسؤولية واحدة بس:

الطبقة	الوصف
Core	(نماذج البيانات)، ال Exceptions، ال Interfaces. مالهش أي اعتماد على أي طبقة ثانية. الطبقة الأساسية: ال Entities
Application	العمل: ال Services، ال DTOs (شكل الطلبات والردود)، ال Interfaces بتاعة الخدمات الخارجية. منطق
Infrastructure	بقاعدة البيانات (DbContext)، ال Repositories، الخدمات الخارجية (Email، تخزين ملفات). التنفيذ الفعلي: الاتصال
API	اللي بتكلم ال Flutter: ال Controllers، ال Middlewares، إعداد المشروع (Program.cs). الطبقة

لمشروع ميزان: اعمل نفس التقسيم بس بأسماء Mizan.API / Mizan.Infrastructure / Mizan.Application / Mizan.Core

2. Repository + Unit of Work

بدل ما كل Service يتكلم مع قاعدة البيانات مباشرة، فيه طبقة وسيطة بتنظم العمليات:

- BaseRepository<T> عام فيه GetByIdAsync, GetAllAsync (مع pagination), AddAsync, Update
- كل Entity ليها Repository خاص بيها بيورث من BaseRepository ويضيف queries خاصة بيه
- IUnitOfWork ال SaveChangesAsync() في مكان واحد، وهو المسؤول الوحيد عن Repositories بيجمع كل ال Repositories بتعمل Lazy (يعني متتنشئش غير لما تتطلب فعلا) — بيوفر أداء

مثال من الريبو — IUnitOfWork

```
public IUserRepository Users => _users ??= new UserRepository(_context);
public ICategoryRepository Categories => _categories ??= new CategoryRepository(_context);

public async Task<int> SaveChangesAsync() => await _context.SaveChangesAsync();
```

لمشروع ميزان: هتحتاج IUnitOfWork فيه ...Users, Contacts, Transactions, Installments, Reminders

3. Entities غنية بمنطق العمل (Rich Domain Model)

(Anemic Model)، كل Entity في الريبو يحتوي على منطق التحقق الخاص بيها جوه نفسها — مش في ال Controller ولا ال Service. بدل ما ال Entity تكون بس خصائص فاضية

- ال Properties كلها private set — متغيرش من برة إلا عن طريق Methods محددة
- إنشاء ال Entity بيتتم عن طريق Factory Method (زي User.CreateManual (...)) مش constructor عادي
- كل تعديل (...UpdateName, ChangePassword) بيعمل validation جوه نفسه ويرمي DomainException لو غلط
- مثال حقيقي: التحقق من رقم الهاتف المصري بيتتم جوه ال Entity نفسه مش في ال Controller

مثال من الريبو — التحقق داخل ال Entity

```
private void SetPhone(string phone)
{
    if (string.IsNullOrWhiteSpace(phone))
        throw new DomainException("Phone number is required");

    phone = phone.Replace(" ", "").Replace("-", "");
    if (phone.StartsWith("+20")) phone = "0" + phone[3..];

    if (phone.Length != 11)
        throw new DomainException("Phone must be 11 digits");

    var validPrefixes = new[] { "010", "011", "012", "015" };
    if (!validPrefixes.Any(p => phone.StartsWith(p)))
        throw new DomainException("Phone must start with 010/011/012/015");

    Phone = phone;
}
```

وعلى منطق حساب remaining_amount في الأقساط — يتحسب جوه ال Entity Installment مش في ال Controller. لمشروع ميزان: نفس الفكرة تنطبق على رقم الواتساب (لازم يبدأ بـ 20 أو 010/011/012/015).

4. الأخطاء الموحدة (Exceptions + Global Middleware)

بدل ما كل Controller يتعامل مع الأخطاء بطريقة، فيه نظام موحد للمشروع كله:

DomainException, NotFoundException, BadRequestException, ForbiddenException
• أنواع Exceptions

• في أي مكان في المشروع Exception بيمسك أي (ExceptionMiddleware) واحد Middleware

• بيحول كل نوع ل HTTP status code مناسب (403, 400, 404...) وشكل JSON موحد

• في بيئة التطوير بيوريك رسالة الخطأ الحقيقية، وفي الإنتاج بيوريك رسالة عامة آمنة

شكل رد الخطأ الموحد في كل المشروع

```
{ "statusCode": 404, "message": "5 'فرع ملاب جتنم لا'" }
```

// نيلكش معديب NotFoundException و :

```
throw new NotFoundException("Transaction not found");
```

```
throw new NotFoundException(nameof(Transaction), id); // دّلويب
```

5. المصادقة (JWT + Refresh Tokens)

- (باسورد ثاني/OTP أطول (لتجديد الدخول من غير Refresh Token + (قصير العمر (للاستخدام اليومي Access Token
- حد أقصى لعدد الأجهزة المتصلة في نفس الوقت (مثلا 5) — لو زاد، أقدم Refresh Token يتلغي تلقائيا
- Rate Limiting بس (5 محاولات في الدقيقة) — يحمي من هجمات تخمين الباسورد Auth ال endpoints على
- CurrentUserId من ال request body مش من ال BaseController، في JWT Claims بيتقرا من ال

مثال من الريبو — BaseController

```
protected int CurrentUserId =>
    int.Parse(User.FindFirstValue(ClaimTypes.NameIdentifier!));

protected IActionResult Success(object? data = null, string? message = null) =>
    Ok(new { success = true, message, data });
```

ميزان: بدل الباسورد، الدخول أسامه OTP عبر واتساب — لكن نفس فكرة ال Rate Limiting وال Refresh Token تتطبق زي ما هي.
لمشروع

6. فحص حالة الحساب بكفاءة (Middleware + Cache)

لسه "مفعّل" في كل request، لكن من غير ما يضغط على قاعدة البيانات في كل مرة — بيستخدم Memory Cache لمدة دقيقتين. الريبو فيه Middleware بيتأكد إن المستخدم

لمشروع ميزان: نفس الفكرة مفيدة لو حد وقف حسابه أو اتحظر — بدل ما تسأل الـ DB في كل request.

7. إعداد قاعدة البيانات بـ Fluent API

- ليها ملف Configuration خاص بيها (`IEntityTypeConfiguration<T>`) — مش Data Annotations على ال Entity
- كل Entity ده بيخلي ال Entity نظيفة ومركزة على منطق العمل بس، والإعدادات التقنية (`MaxLength`, `Index`, `Unique`) في مكان تاني
- فيه استخدام لـ `Owned Entity` (زي `Address`) — مفيد لو عندك بيانات مركبة زي عنوان المحل

مثال من الريبو — `UserConfiguration`

```
builder.Property(u => u.FirstName).IsRequired().HasMaxLength(50);
builder.HasIndex(u => u.Email).IsUnique();
builder.Property(u => u.IsActive).IsRequired().HasDefaultValue(true);
```

8. التحقق من المدخلات (DTOs + DataAnnotations)

كل Request بيتحدد شكله في DTO منفصل، مع رسائل خطأ بالعربي مباشرة:

```
public class RegisterRequest
{
    [Required(ErrorMessage = "First name is required")]
    [MaxLength(50, ErrorMessage = "First name max 50 chars")]
    public string FirstName { get; set; } = string.Empty;

    [Required(ErrorMessage = "Phone number is required")]
    public string Phone { get; set; } = string.Empty;
}
```

ده بالظبط سبب طلبك تقسيم `first_name/last_name` في ال ERD — ده معيار موجود فعليا في الريبو ده.

لمشروع ميزان /speckit.constitution لما تكتب — 9. checklist

انسخ ده واستخدمه كأساس لل constitution بتاعك في Antigravity:

مخزنه Clean Architecture عراب Mizan.Core, Mizan.Application, Mizan.Infrastructure, Mizan.API – ميسرت سقنب .

- Repository + IUnitOfWork لكل لوصول (lazy-loaded repos)
- Entity ال هوج Entities لمعلا قطنمب فينغ private setters + factory methods + validation
- Exceptions (DomainException, NotFoundException, BadRequestException, ForbiddenException) مصمخ Middleware عم HTTP responses ل ملوحي دحاو
- BaseController هي Success()/Created() helpers و CurrentUserId نم JWT
- JWT Access + Refresh Token, ددغل صقأ دح عم
- Rate Limiting سب /auth endpoints (5 سب (ةقيد/تالواحم 5)
- EF Core Fluent API (IEntityTypeConfiguration) Entity, شم Data Annotations اهلع
- DTOs Request عم DataAnnotations ل لكل ةلمفم
- FK و id ال لك (IDENTITY) نم FK و id ال لك